

IMPLEMENTATION SUMMARY

Assessment: 3

Class: Software Technology 1 (4483)

Group: Group 3

Authors: u3320473 | u3218103 | u3313879

Date: 20 May 2025

PROJECT TITLE AND GROUP MEMBERS

Project Title: Macroinvertebrate Image Analysis System

Group Number: Group 3 Software Technology 1 (4483)

Members:

- u3320473: Backend & Data Architect
- u3218103: Visualisation & Image Specialist
- u3313879: GUI Developer & Integrator

BRIEF PROJECT GOAL

To build a modular Python desktop application that loads, indexes, and analyses a dataset of 2,665 freshwater macroinvertebrate images across 17 species. The application performs Stage 1 Exploratory Data Analysis and provides a Stage 3 Tkinter GUI for interactive exploration of the dataset.

SYSTEM DESIGN OVERVIEW

The application is organised into three logical layers:

The Data Layer: handles image loading, indexing, and structured storage. The DatasetIndexer scans the dataset folder recursively using pathlib, reads each image with OpenCV, creates ImageRecord objects, and returns a Pandas DataFrame. A species filter parameter allows selective indexing to avoid loading the entire dataset on every run.

The EDA Layer: handles chart generation and statistical analysis. The EDAService takes the DataFrame produced by DatasetIndexer and generates ten charts and a CSV summary report using Matplotlib and OpenCV, saving all outputs to the outputs/eda/ directory which is created automatically if missing.

The GUI Layer: handles user interaction through a Tkinter desktop application. MacroApp integrates DatasetIndexer and EDAService,

providing a stats panel, species dropdown, image viewer, chart viewer, and species count table. A SpeciesSelector dialog runs before the main app to allow targeted species loading.

CLASS AND MODULE OVERVIEW

Class File Responsibility

1. ImageRecord src/models/records.py Dataclass storing metadata for one image file
2. DatasetIndexer src/services/ Scans dataset folder, dataset_indexer.py builds indexed DataFrame
3. EDAService src/services/eda_service.py Generates charts and CSV from indexed DataFrame
4. SpeciesSelector src/species_selector.py Startup dialog for selecting species to load
5. MacroApp src/App_SecondVer.py Tkinter desktop GUI, integrates all components

Supporting files:

src/config.py central paths and constants
src/main.py backend verification script
.vscode/launch.json VS Code run configuration

Libraries used:

Pandas Stores indexed image records as a structured DataFrame for analysis and filtering

Numpy Numerical array operations on image data

opencv-python Reads and decodes image files from disk, converts colour channels for display

matplotlib Generates and saves all ten EDA charts as PNG files

Pillow Converts OpenCV images into a format compatible with Tkinter for display

Tkinter Builds the desktop GUI without additional installation

pathlib Handles file paths cleanly across Windows, Mac, and Linux

KEY FEATURES IMPLEMENTED

- Species selection dialog on startup with minimum 3 species requirement
- Recursive dataset scanning supporting .jpg, .jpeg, .png, .bmp, .tiff
- Pandas DataFrame with one row per image for structured analysis
- Ten EDA charts: class distribution bar chart, category pie chart, width histogram, height histogram, aspect ratio histogram, channels distribution, width vs height scatter plot, dimension boxplot, sample image grid, top 5 species chart
- CSV statistical summary report exported via pandas describe()
- Automatic creation of outputs/eda/ directory if missing
- Interactive GUI: stats panel, species image browser, chart viewer, species count table
- Error handling for missing dataset, invalid images, and pre-generation chart viewing attempts

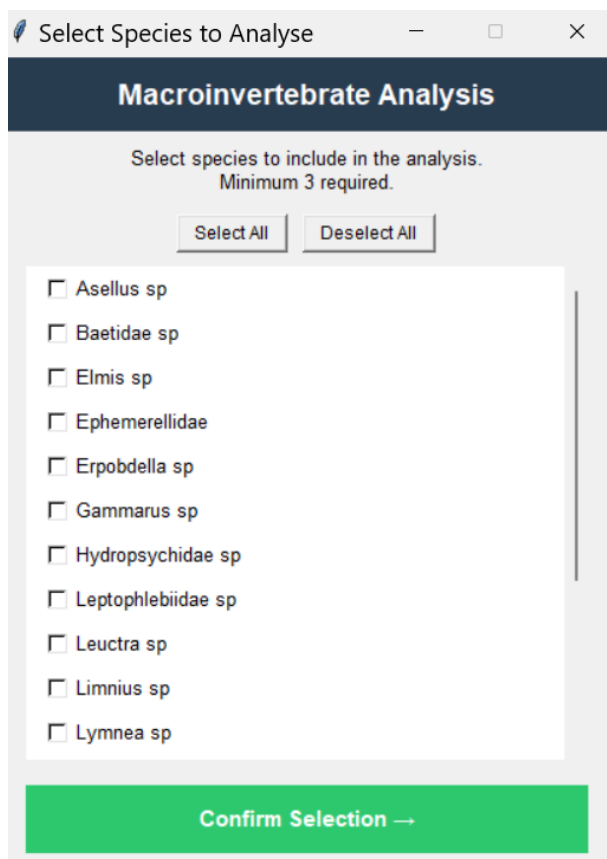
TESTING SUMMARY

Manual testing was conducted across eleven scenarios documented in MANUAL_TESTING.md. Key scenarios included:

- Selecting fewer than 3 species at startup: warning popup shown
- Closing species selector without confirming: app exits cleanly
- Clicking View Chart before generating: warning popup shown
- Missing dataset folder: startup error popup with clear message
- Full end-to-end run with selected species: all features passed

SCREENSHOTS / SAMPLE OUTPUTS

Species selector Dialog + Main GUI:



Selecting less than 3:

Macroinvertebrate Analysis

Too Few Selected



Please select at least 3 species.
You selected 1.

OK



Erpobdella sp

Gammarus sp

Hydropsychidae sp

Leptophlebiidae sp

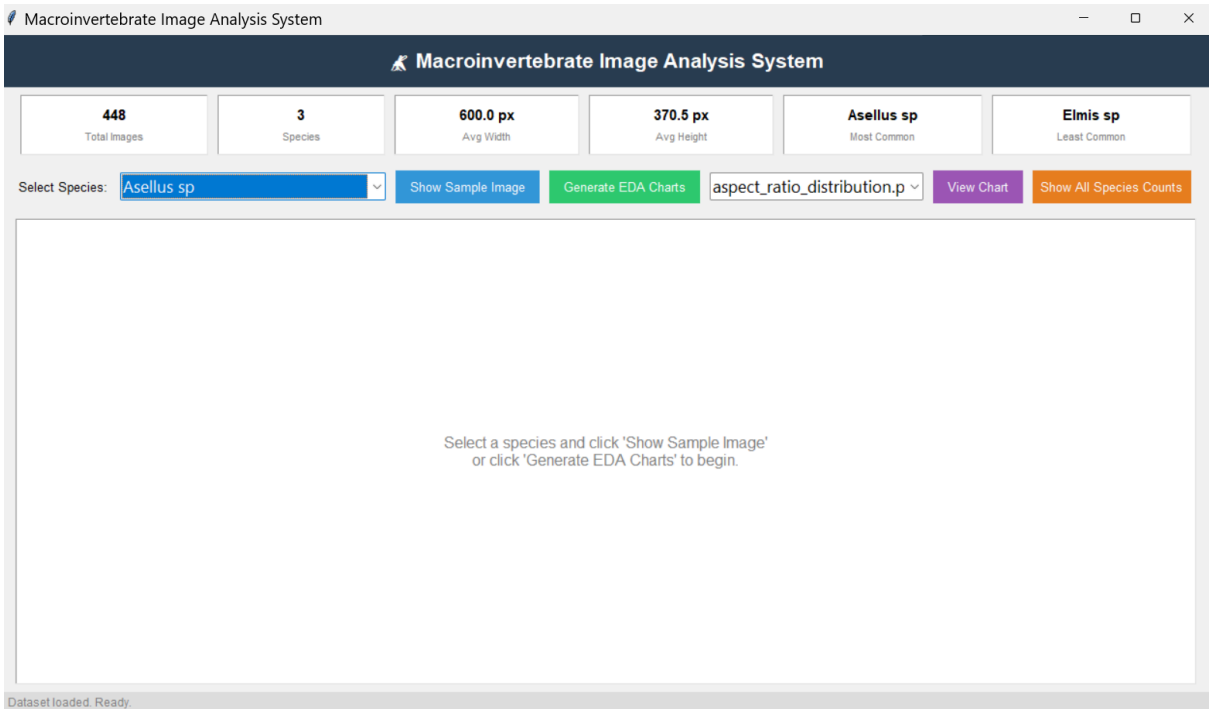
Leuctra sp

Limnius sp

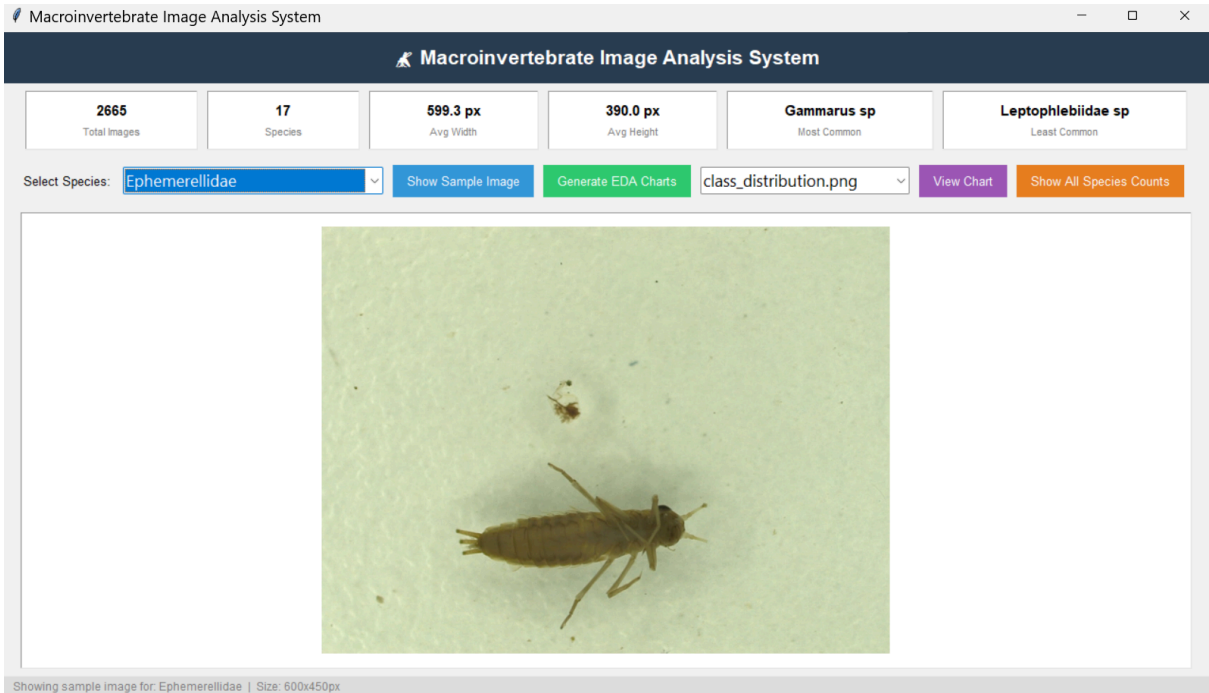
Lymnea sp

Confirm Selection →

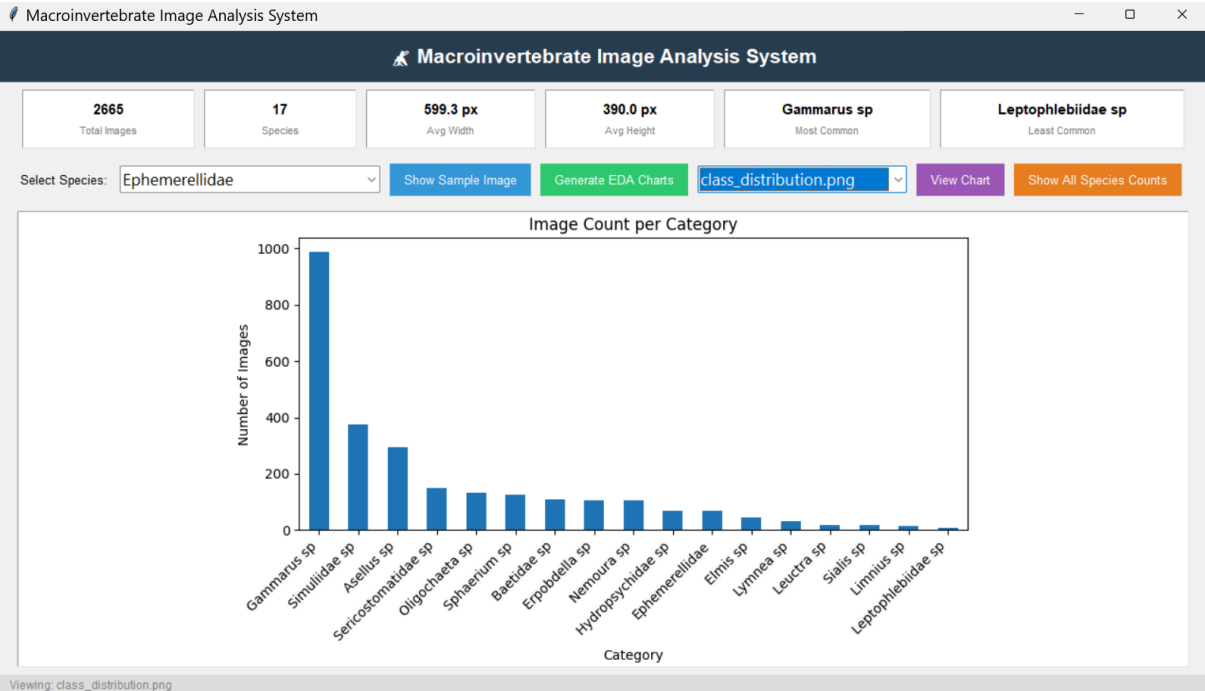
Selecting only three:



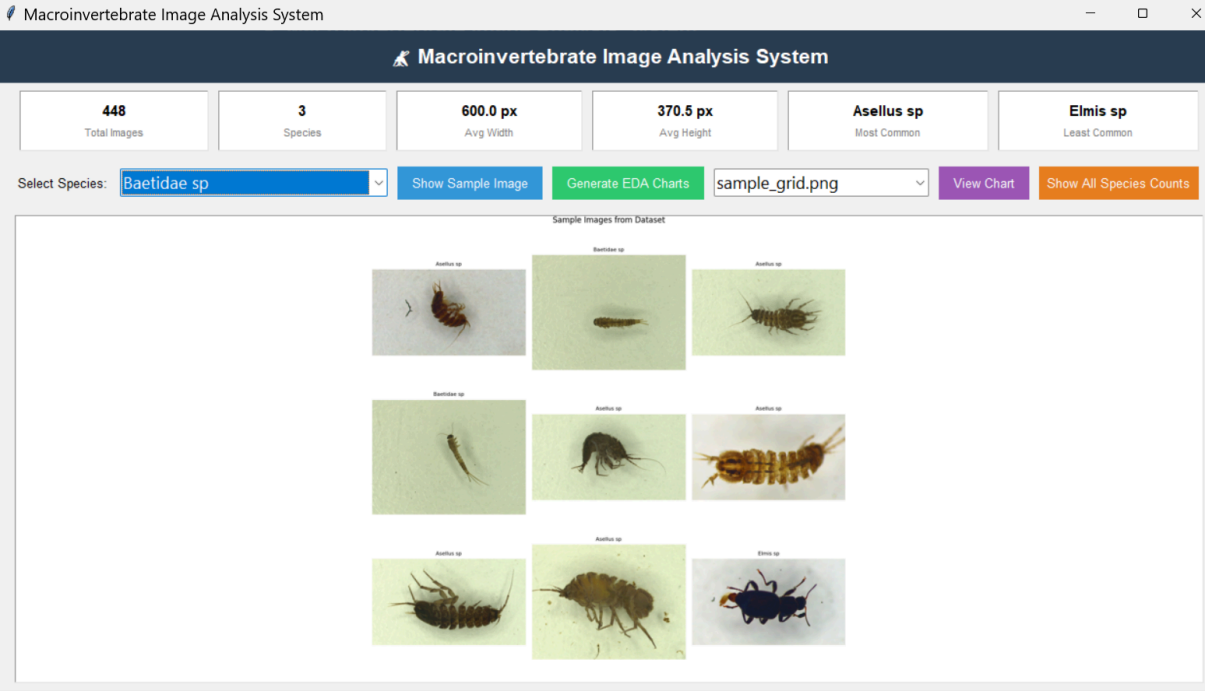
Selecting all species:



Class Distribution chart:



Sample Grid:



Species count table:



ACKNOWLEDGEMENT OF REUSED OR ADAPTED CODE

The general structure of the DatasetIndexer class was informed by the dataset indexing pattern provided in the Assignment 3 Full Guidance document supplied by the unit [1]. The implementation was extended to include species filtering, a list-based record store, and the get_summary() method.

The Tkinter GUI structure was developed independently. The chart generation methods in EDAService were written by u3218103 using Matplotlib documentation [2] and OpenCV documentation [3] as references.

All reused or adapted patterns are acknowledged in inline comments within the relevant files.

WORK DIVISION SUMMARY

u3320473: Built the data foundation: config.py, records.py, dataset_indexer.py, species_selector.py, and main.py. Responsible for backend integration, README, and MANUAL_TESTING documentation.

u3218103: Built eda_service.py including all ten chart generation

methods and the CSV export. Responsible for all EDA outputs and visual analysis.

u3313879: Built App_SecondVer.py including the full Tkinter interface. Responsible for integrating DatasetIndexer and EDAService into the GUI and managing the GitHub repository.

The Species Selector was co-developed by u3313879 and u3320473

All code was coordinated through a shared GitHub repository. Final integration and testing were completed collaboratively.

REFERENCES

- [1] University of Canberra, "Assignment 3 Full Guidance and Coding Examples," Software Technology 1 (4483/8995), 2025.
- [2] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," Computing in Science & Engineering, vol. 9, no. 3, pp. 90-95, 2007.
[Online]. Available: <https://matplotlib.org>
- [3] OpenCV Team, "OpenCV Documentation," 2025. [Online].
Available: <https://docs.opencv.org>
- [4] The pandas development team, "pandas: Powerful Python Data Analysis Toolkit," 2025. [Online].
Available: <https://pandas.pydata.org>
- [5] K. T. M., "Stream Macroinvertebrates Dataset," Kaggle, 2021.
[Online]. Available:
<https://www.kaggle.com/datasets/kennethm/stream-macroinvertebrates>
- [6] Python Software Foundation, "tkinter — Python interface to Tcl/Tk," Python 3 Documentation, 2025. [Online].
Available: <https://docs.python.org/3/library/tkinter.html>
- [7] Python Imaging Library contributors, "Pillow Documentation," 2025. [Online]. Available: <https://pillow.readthedocs.io>